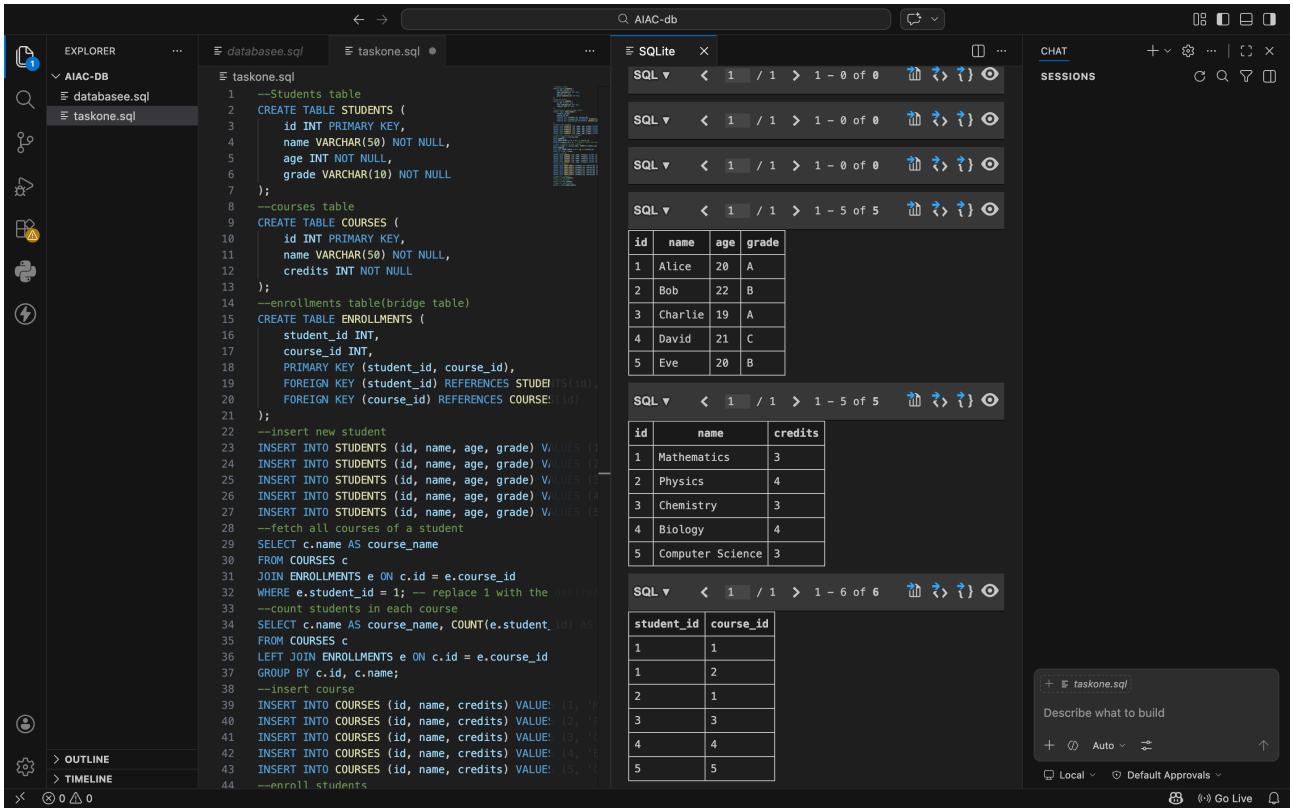
AIAC ASSIGNMENT - 16

NAME: ISHA MADEEHA
HALLTICKET NO: 2303A51017
BATCH - 01

TASK-1:




```sql
--Students table
CREATE TABLE STUDENTS (
    id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    age INT NOT NULL,
    grade VARCHAR(10) NOT NULL
);
--courses table
CREATE TABLE COURSES (
    id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    credits INT NOT NULL
);
--enrollments table(bridge table)
CREATE TABLE ENROLLMENTS (
    student_id INT,
    course_id INT,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES STUDE
    FOREIGN KEY (course_id) REFERENCES COURSE
);
--insert new student
INSERT INTO STUDENTS (id, name, age, grade) V
INSERT INTO STUDENTS (id, name, age, grade) V
INSERT INTO STUDENTS (id, name, age, grade) V
INSERT INTO STUDENTS (id, name, age, grade) V
INSERT INTO STUDENTS (id, name, age, grade) V
--fetch all courses of a student
SELECT c.name AS course_name
FROM COURSES c
JOIN ENROLLMENTS e ON c.id = e.course_id
WHERE e.student_id = 1; -- replace 1 with the
--count students in each course
SELECT c.name AS course_name, COUNT(e.student_
FROM COURSES c
LEFT JOIN ENROLLMENTS e ON c.id = e.course_id
GROUP BY c.id, c.name;
--insert course
INSERT INTO COURSES (id, name, credits) VALUE
INSERT INTO COURSES (id, name, credits) VALUE
INSERT INTO COURSES (id, name, credits) VALUE
INSERT INTO COURSES (id, name, credits) VALUE
INSERT INTO COURSES (id, name, credits) VALUE
--enroll students
```

SQL ▾ 1 / 1 1 – 0 of 0

SQL ▾ 1 / 1 1 – 0 of 0

SQL ▾ 1 / 1 1 – 0 of 0

SQL ▾ 1 / 1 1 – 5 of 5

id	name	age	grade
1	Alice	20	A
2	Bob	22	B
3	Charlie	19	A
4	David	21	C
5	Eve	20	B

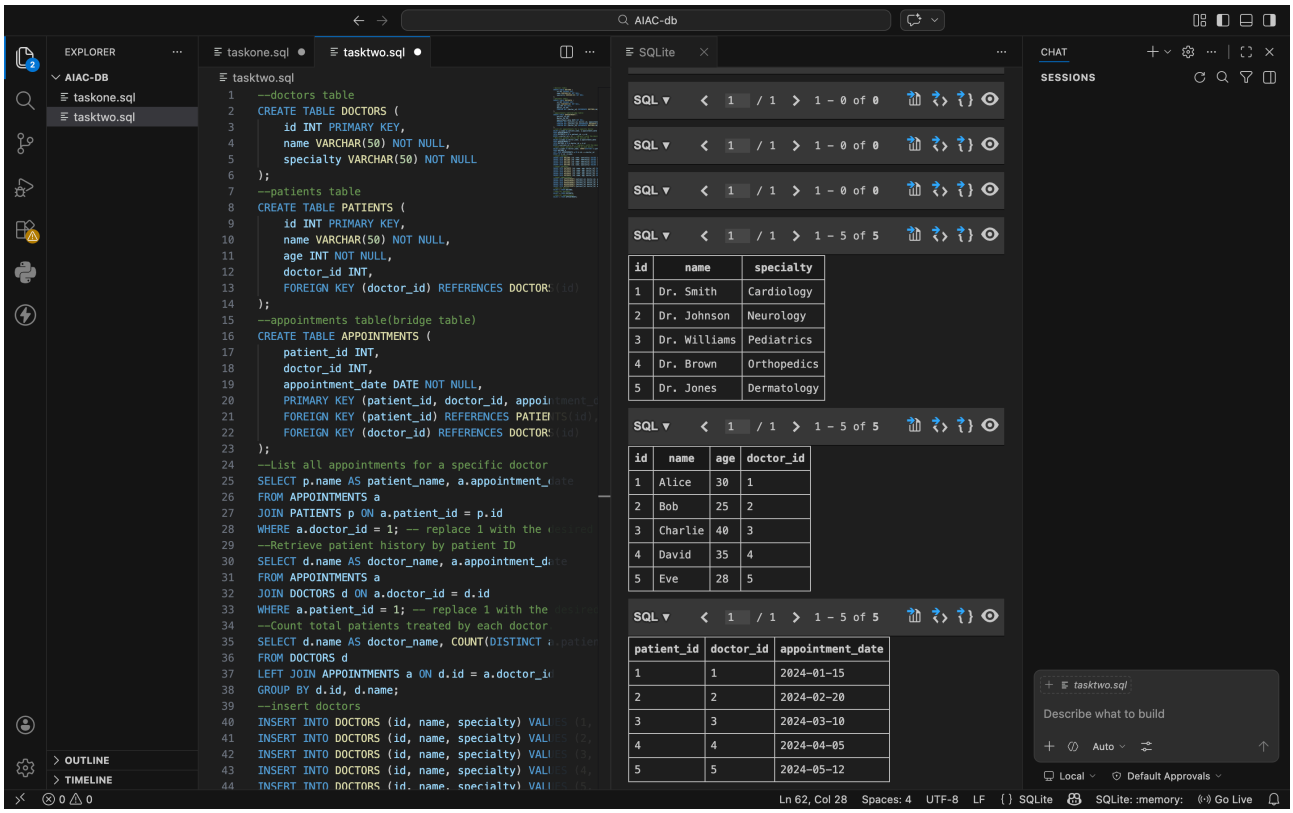
SQL ▾ 1 / 1 1 – 5 of 5

id	name	credits
1	Mathematics	3
2	Physics	4
3	Chemistry	3
4	Biology	4
5	Computer Science	3

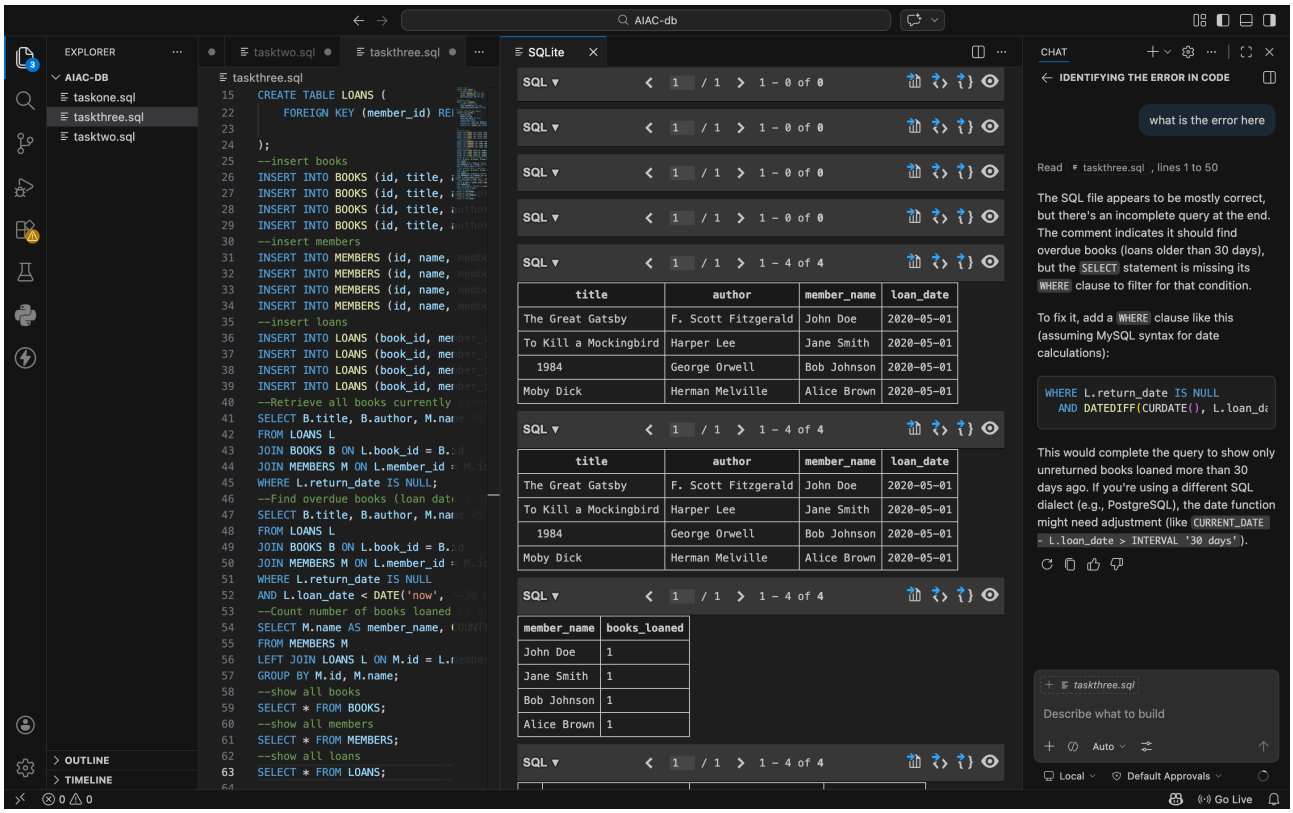
SQL ▾ 1 / 1 1 – 6 of 6

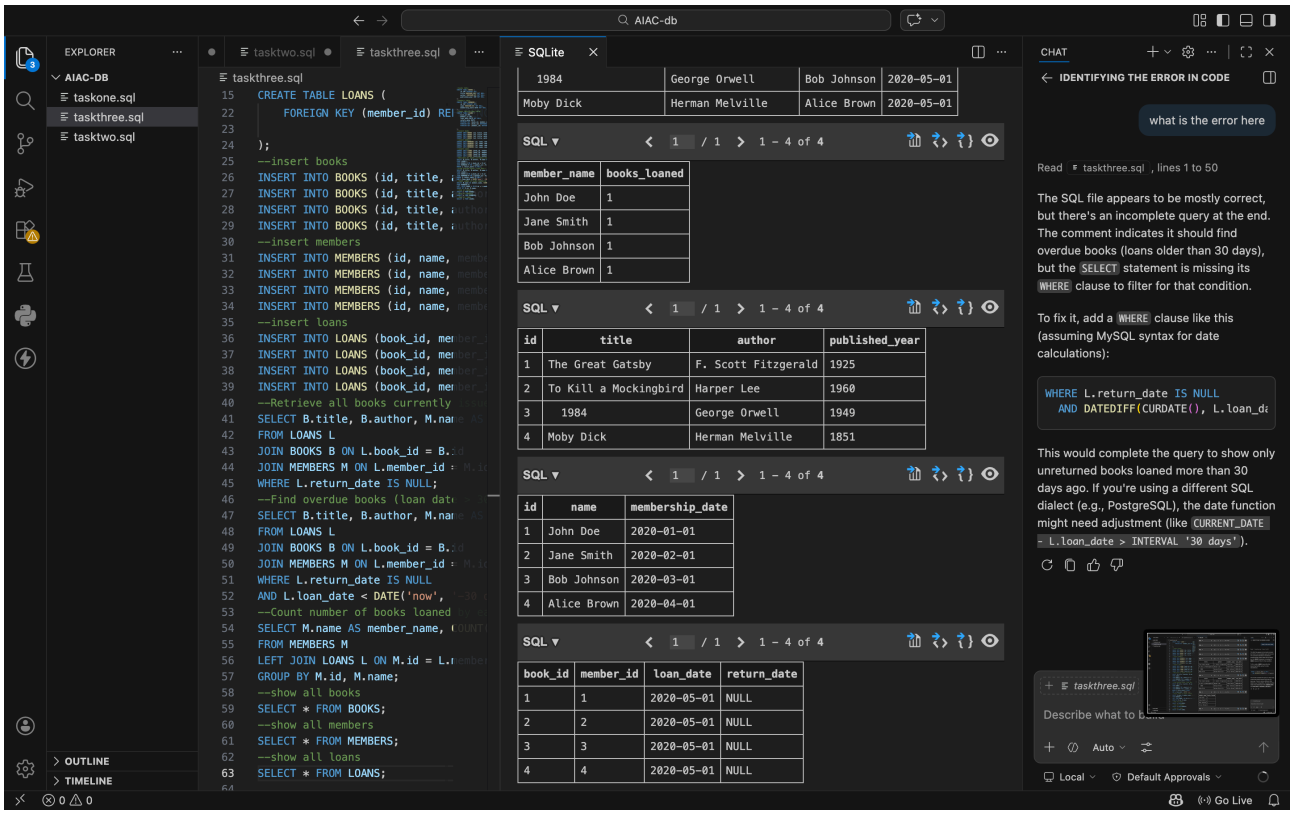
student_id	course_id
1	1
1	2
2	1
3	3
4	4
5	5

TASK - 2:

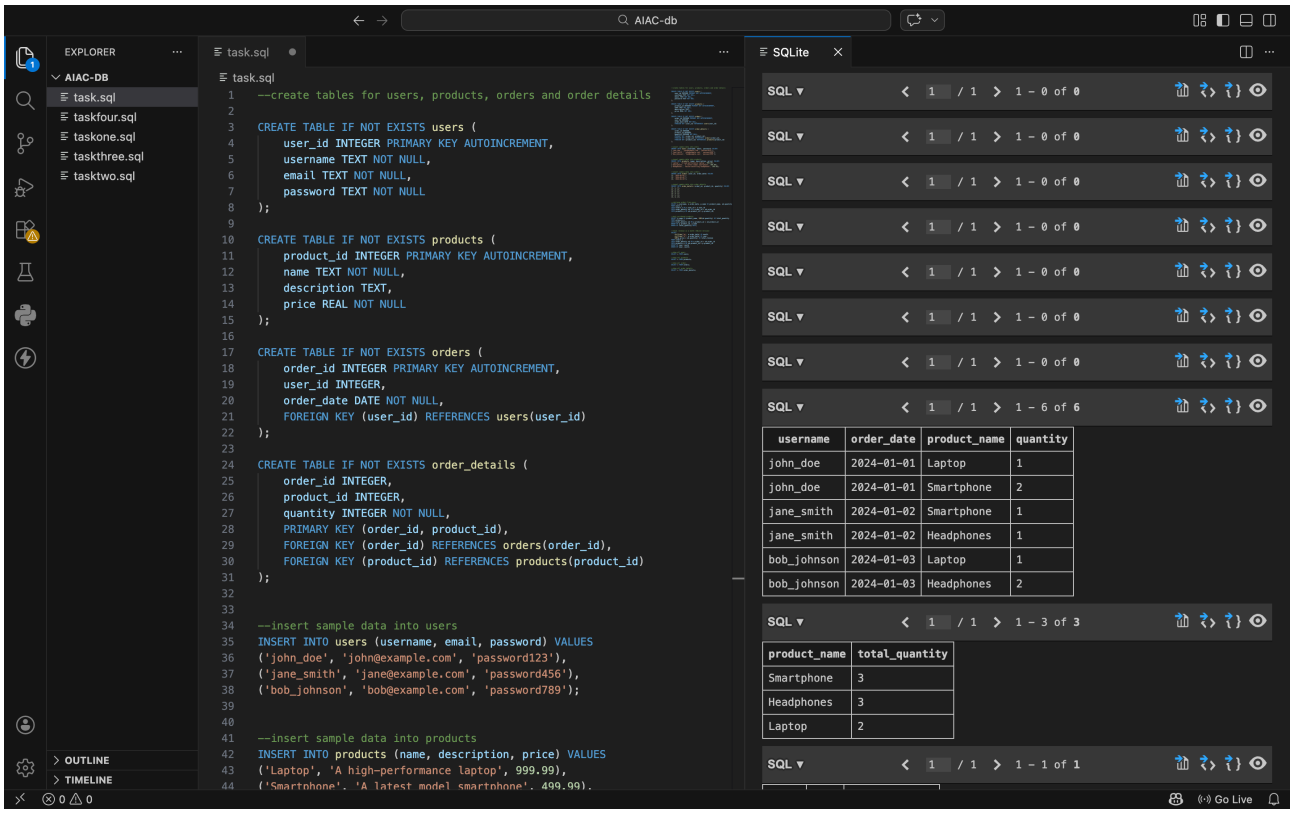

AIAC-db
EXPLORER
AIAC-DB
taskone.sql
tasktwo.sql
tasktwo.sql
--doctors table
CREATE TABLE DOCTORS (
id INT PRIMARY KEY,
name VARCHAR(50) NOT NULL,
specialty VARCHAR(50) NOT NULL
);
--patients table
CREATE TABLE PATIENTS (
id INT PRIMARY KEY,
name VARCHAR(50) NOT NULL,
age INT NOT NULL,
doctor_id INT,
FOREIGN KEY (doctor_id) REFERENCES DOCTOR
);
--appointments table(bridge table)
CREATE TABLE APPOINTMENTS (
patient_id INT,
doctor_id INT,
appointment_date DATE NOT NULL,
PRIMARY KEY (patient_id, doctor_id, appoi
FOREIGN KEY (patient_id) REFERENCES PATIE
FOREIGN KEY (doctor_id) REFERENCES DOCTOR
);
--List all appointments for a specific doctor
SELECT p.name AS patient_name, a.appointment_
FROM APPOINTMENTS a
JOIN PATIENTS p ON a.patient_id = p.id
WHERE a.doctor_id = 1; -- replace 1 with the
--Retrieve patient history by patient ID
SELECT d.name AS doctor_name, a.appointment_d
FROM APPOINTMENTS a
JOIN DOCTORS d ON a.doctor_id = d.id
WHERE a.patient_id = 1; -- replace 1 with the
--Count total patients treated by each doctor
SELECT d.name AS doctor_name, COUNT(DISTINCT
FROM DOCTORS d
LEFT JOIN APPOINTMENTS a ON d.id = a.doctor_i
GROUP BY d.id, d.name;
--insert doctors
INSERT INTO DOCTORS (id, name, specialty) VALU
INSERT INTO DOCTORS (id, name, specialty) VALU
INSERT INTO DOCTORS (id, name, specialty) VALU
INSERT INTO DOCTORS (id, name, specialty) VALU
INSERT INTO DOCTORS (id, name, specialty) VALU
OUTLINE
TIMELINE
SQLite
SQL ▾ 1 / 1 1 – 0 of 0
SQL ▾ 1 / 1 1 – 0 of 0
SQL ▾ 1 / 1 1 – 0 of 0
SQL ▾ 1 / 1 1 – 5 of 5
id name specialty
1 Dr. Smith Cardiology
2 Dr. Johnson Neurology
3 Dr. Williams Pediatrics
4 Dr. Brown Orthopedics
5 Dr. Jones Dermatology
SQL ▾ 1 / 1 1 – 5 of 5
id name age doctor_id
1 Alice 30 1
2 Bob 25 2
3 Charlie 40 3
4 David 35 4
5 Eve 28 5
SQL ▾ 1 / 1 1 – 5 of 5
patient_id doctor_id appointment_date
1 1 2024-01-15
2 2 2024-02-20
3 3 2024-03-10
4 4 2024-04-05
5 5 2024-05-12
CHAT
SESSIONS
tasktwo.sql
Describe what to build
Auto
Local
Default Approvals
Ln 62, Col 28 Spaces: 4 UTF-8 LF SQLite SQLite: :memory: Go Live

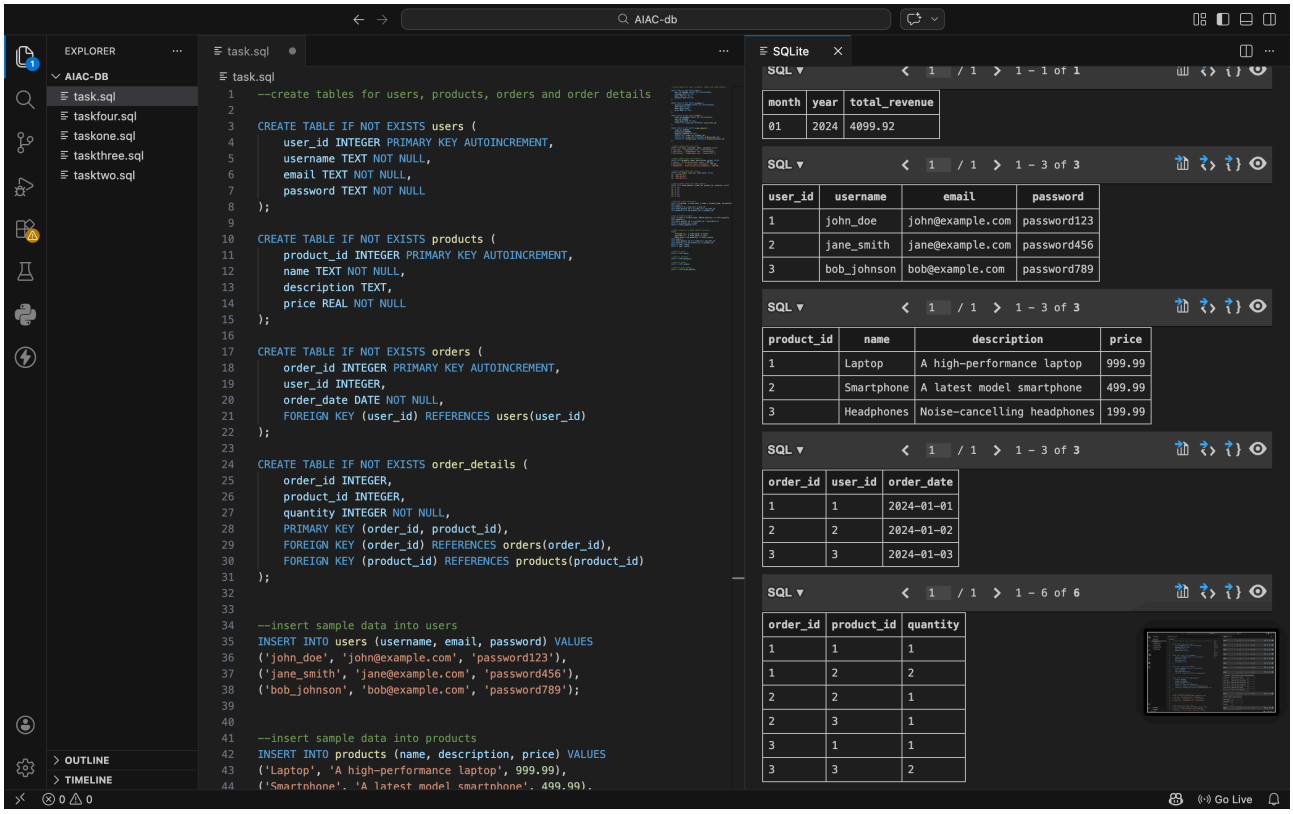
TASK - 3:


AIAC-db
EXPLORER
AIAC-DB
taskone.sql
taskthree.sql
tasktwo.sql
tasktwo.sql
taskthree.sql
taskthree.sql
15 CREATE TABLE LOANS (
22 FOREIGN KEY (member_id) REI
23
24);
25 --insert books
26 INSERT INTO BOOKS (id, title,
27 INSERT INTO BOOKS (id, title,
28 INSERT INTO BOOKS (id, title,
29 INSERT INTO BOOKS (id, title,
30 --insert members
31 INSERT INTO MEMBERS (id, name,
32 INSERT INTO MEMBERS (id, name,
33 INSERT INTO MEMBERS (id, name,
34 INSERT INTO MEMBERS (id, name,
35 --insert loans
36 INSERT INTO LOANS (book_id, me
37 INSERT INTO LOANS (book_id, me
38 INSERT INTO LOANS (book_id, me
39 INSERT INTO LOANS (book_id, me
40 --Retrieve all books currently
41 SELECT B.title, B.author, M.na
42 FROM LOANS L
43 JOIN BOOKS B ON L.book_id = B.
44 JOIN MEMBERS M ON L.member_id
45 WHERE L.return_date IS NULL;
46 --Find overdue books (loan dat
47 SELECT B.title, B.author, M.na
48 FROM LOANS L
49 JOIN BOOKS B ON L.book_id = B.
50 JOIN MEMBERS M ON L.member_id
51 WHERE L.return_date IS NULL
52 AND L.loan_date < DATE('now',
53 --Count number of books loaned
54 SELECT M.name AS member_name,
55 FROM MEMBERS M
56 LEFT JOIN LOANS L ON M.id = L.
57 GROUP BY M.id, M.name;
58 --show all books
59 SELECT * FROM BOOKS;
60 --show all members
61 SELECT * FROM MEMBERS;
62 --show all loans
63 SELECT * FROM LOANS;
OUTLINE
TIMELINE
SQLite
SQL ▾ 1 / 1 1 – 0 of 0
SQL ▾ 1 / 1 1 – 0 of 0
SQL ▾ 1 / 1 1 – 0 of 0
SQL ▾ 1 / 1 1 – 0 of 0
SQL ▾ 1 / 1 1 – 4 of 4
title | author | member_name | loan_date
The Great Gatsby | F. Scott Fitzgerald | John Doe | 2020-05-01
To Kill a Mockingbird | Harper Lee | Jane Smith | 2020-05-01
1984 | George Orwell | Bob Johnson | 2020-05-01
Moby Dick | Herman Melville | Alice Brown | 2020-05-01
SQL ▾ 1 / 1 1 – 4 of 4
title | author | member_name | loan_date
The Great Gatsby | F. Scott Fitzgerald | John Doe | 2020-05-01
To Kill a Mockingbird | Harper Lee | Jane Smith | 2020-05-01
1984 | George Orwell | Bob Johnson | 2020-05-01
Moby Dick | Herman Melville | Alice Brown | 2020-05-01
SQL ▾ 1 / 1 1 – 4 of 4
member_name | books_loaned
John Doe | 1
Jane Smith | 1
Bob Johnson | 1
Alice Brown | 1
SQL ▾ 1 / 1 1 – 4 of 4
CHAT
IDENTIFYING THE ERROR IN CODE
what is the error here
Read taskthree.sql , lines 1 to 50
The SQL file appears to be mostly correct, but there's an incomplete query at the end. The comment indicates it should find overdue books (loans older than 30 days), but the SELECT statement is missing its WHERE clause to filter for that condition.
To fix it, add a WHERE clause like this (assuming MySQL syntax for date calculations):
WHERE L.return_date IS NULL
AND DATEDIFF(CURDATE(), L.loan_da
This would complete the query to show only unreturned books loaned more than 30 days ago. If you're using a different SQL dialect (e.g., PostgreSQL), the date function might need adjustment (like CURRENT_DATE - L.loan_date > INTERVAL '30 days').
taskthree.sql
Describe what to build
Auto
Local
Default Approvals
0 0
Go Live

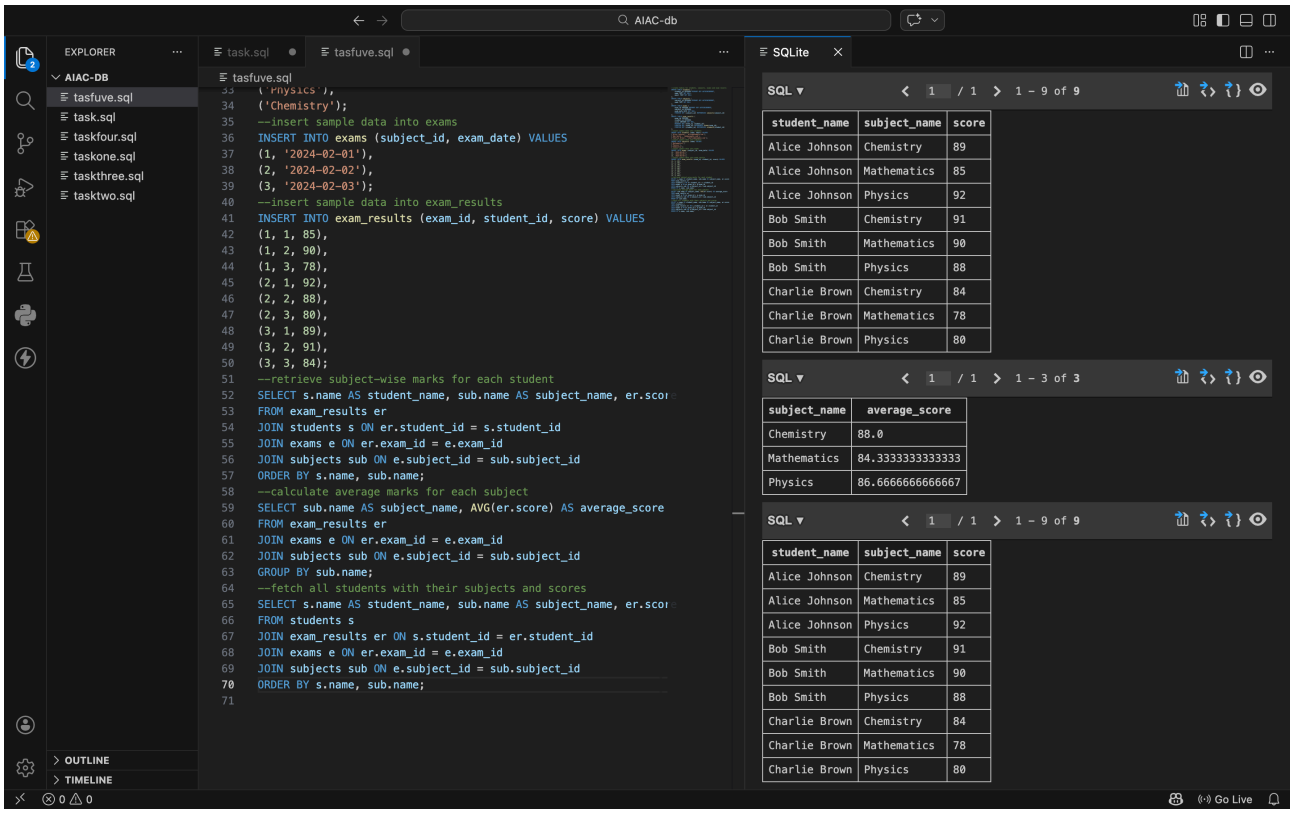

AIAC-db
EXPLORER
AIAC-DB
taskone.sql
taskthree.sql
tasktwo.sql
taskthree.sql
SQLite
1984 | George Orwell | Bob Johnson | 2020-05-01
Moby Dick | Herman Melville | Alice Brown | 2020-05-01
SQL ▾ 1 / 1 1 – 4 of 4
member_name | books_loaned
John Doe | 1
Jane Smith | 1
Bob Johnson | 1
Alice Brown | 1
SQL ▾ 1 / 1 1 – 4 of 4
id | title | author | published_year
1 | The Great Gatsby | F. Scott Fitzgerald | 1925
2 | To Kill a Mockingbird | Harper Lee | 1960
3 | 1984 | George Orwell | 1949
4 | Moby Dick | Herman Melville | 1851
SQL ▾ 1 / 1 1 – 4 of 4
id | name | membership_date
1 | John Doe | 2020-01-01
2 | Jane Smith | 2020-02-01
3 | Bob Johnson | 2020-03-01
4 | Alice Brown | 2020-04-01
SQL ▾ 1 / 1 1 – 4 of 4
book_id | member_id | loan_date | return_date
1 | 1 | 2020-05-01 | NULL
2 | 2 | 2020-05-01 | NULL
3 | 3 | 2020-05-01 | NULL
4 | 4 | 2020-05-01 | NULL
CHAT
IDENTIFYING THE ERROR IN CODE
what is the error here
Read taskthree.sql , lines 1 to 50
The SQL file appears to be mostly correct, but there's an incomplete query at the end. The comment indicates it should find overdue books (loans older than 30 days), but the SELECT statement is missing its WHERE clause to filter for that condition.
To fix it, add a WHERE clause like this (assuming MySQL syntax for date calculations):
WHERE L.return_date IS NULL
AND DATEDIFF(CURDATE(), L.loan_da
This would complete the query to show only unreturned books loaned more than 30 days ago. If you're using a different SQL dialect (e.g., PostgreSQL), the date function might need adjustment (like CURRENT_DATE - L.loan_date > INTERVAL '30 days').
taskthree.sql
Auto
Local
Default Approvals
Go Live

TASK - 4:


AIAC-db
EXPLORER
AIAC-DB
task.sql
taskfour.sql
taskone.sql
taskthree.sql
tasktwo.sql
OUTLINE
TIMELINE
task.sql
--create tables for users, products, orders and order details
CREATE TABLE IF NOT EXISTS users (
user_id INTEGER PRIMARY KEY AUTOINCREMENT,
username TEXT NOT NULL,
email TEXT NOT NULL,
password TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS products (
product_id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT NOT NULL,
description TEXT,
price REAL NOT NULL
);
CREATE TABLE IF NOT EXISTS orders (
order_id INTEGER PRIMARY KEY AUTOINCREMENT,
user_id INTEGER,
order_date DATE NOT NULL,
FOREIGN KEY (user_id) REFERENCES users(user_id)
);
CREATE TABLE IF NOT EXISTS order_details (
order_id INTEGER,
product_id INTEGER,
quantity INTEGER NOT NULL,
PRIMARY KEY (order_id, product_id),
FOREIGN KEY (order_id) REFERENCES orders(order_id),
FOREIGN KEY (product_id) REFERENCES products(product_id)
);
--insert sample data into users
INSERT INTO users (username, email, password) VALUES
('john_doe', 'john@example.com', 'password123'),
('jane_smith', 'jane@example.com', 'password456'),
('bob_johnson', 'bob@example.com', 'password789');
--insert sample data into products
INSERT INTO products (name, description, price) VALUES
('Laptop', 'A high-performance laptop', 999.99),
('Smartphone', 'A latest model smartphone', 499.99),
SQLite
SQL ▾ 1 / 1 1 – 0 of 0
SQL ▾ 1 / 1 1 – 6 of 6
username order_date product_name quantity
john_doe 2024-01-01 Laptop 1
john_doe 2024-01-01 Smartphone 2
jane_smith 2024-01-02 Smartphone 1
jane_smith 2024-01-02 Headphones 1
bob_johnson 2024-01-03 Laptop 1
bob_johnson 2024-01-03 Headphones 2
SQL ▾ 1 / 1 1 – 3 of 3
product_name total_quantity
Smartphone 3
Headphones 3
Laptop 2
SQL ▾ 1 / 1 1 – 1 of 1
Go Live


AIAC-db
EXPLORER
AIAC-DB
task.sql
taskfour.sql
taskone.sql
taskthree.sql
tasktwo.sql
OUTLINE
TIMELINE
task.sql
--create tables for users, products, orders and order details
CREATE TABLE IF NOT EXISTS users (
user_id INTEGER PRIMARY KEY AUTOINCREMENT,
username TEXT NOT NULL,
email TEXT NOT NULL,
password TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS products (
product_id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT NOT NULL,
description TEXT,
price REAL NOT NULL
);
CREATE TABLE IF NOT EXISTS orders (
order_id INTEGER PRIMARY KEY AUTOINCREMENT,
user_id INTEGER,
order_date DATE NOT NULL,
FOREIGN KEY (user_id) REFERENCES users(user_id)
);
CREATE TABLE IF NOT EXISTS order_details (
order_id INTEGER,
product_id INTEGER,
quantity INTEGER NOT NULL,
PRIMARY KEY (order_id, product_id),
FOREIGN KEY (order_id) REFERENCES orders(order_id),
FOREIGN KEY (product_id) REFERENCES products(product_id)
);
--insert sample data into users
INSERT INTO users (username, email, password) VALUES
('john_doe', 'john@example.com', 'password123'),
('jane_smith', 'jane@example.com', 'password456'),
('bob_johnson', 'bob@example.com', 'password789');
--insert sample data into products
INSERT INTO products (name, description, price) VALUES
('Laptop', 'A high-performance laptop', 999.99),
('Smartphone', 'A latest model smartphone', 499.99),
SQLite
SQL ▾ 1 / 1 1 – 1 of 1
month year total_revenue
01 2024 4099.92
SQL ▾ 1 / 1 1 – 3 of 3
user_id username email password
1 john_doe john@example.com password123
2 jane_smith jane@example.com password456
3 bob_johnson bob@example.com password789
SQL ▾ 1 / 1 1 – 3 of 3
product_id name description price
1 Laptop A high-performance laptop 999.99
2 Smartphone A latest model smartphone 499.99
3 Headphones Noise-cancelling headphones 199.99
SQL ▾ 1 / 1 1 – 3 of 3
order_id user_id order_date
1 1 2024-01-01
2 2 2024-01-02
3 3 2024-01-03
SQL ▾ 1 / 1 1 – 6 of 6
order_id product_id quantity
1 1 1
1 2 2
2 2 1
2 3 1
3 1 1
3 3 2
Go Live

TASK - 5:


AIAC-db
EXPLORER
AIAC-DB
tasfuve.sql
task.sql
taskfour.sql
taskone.sql
taskthree.sql
tasktwo.sql
OUTLINE
TIMELINE
task.sql
tasfuve.sql
tasfuve.sql
('Physics'),
('Chemistry');
--insert sample data into exams
INSERT INTO exams (subject_id, exam_date) VALUES
(1, '2024-02-01'),
(2, '2024-02-02'),
(3, '2024-02-03');
--insert sample data into exam_results
INSERT INTO exam_results (exam_id, student_id, score) VALUES
(1, 1, 85),
(1, 2, 90),
(1, 3, 78),
(2, 1, 92),
(2, 2, 88),
(2, 3, 80),
(3, 1, 89),
(3, 2, 91),
(3, 3, 84);
--retrieve subject-wise marks for each student
SELECT s.name AS student_name, sub.name AS subject_name, er.scor
FROM exam_results er
JOIN students s ON er.student_id = s.student_id
JOIN exams e ON er.exam_id = e.exam_id
JOIN subjects sub ON e.subject_id = sub.subject_id
ORDER BY s.name, sub.name;
--calculate average marks for each subject
SELECT sub.name AS subject_name, AVG(er.score) AS average_score
FROM exam_results er
JOIN exams e ON er.exam_id = e.exam_id
JOIN subjects sub ON e.subject_id = sub.subject_id
GROUP BY sub.name;
--fetch all students with their subjects and scores
SELECT s.name AS student_name, sub.name AS subject_name, er.scor
FROM students s
JOIN exam_results er ON s.student_id = er.student_id
JOIN exams e ON er.exam_id = e.exam_id
JOIN subjects sub ON e.subject_id = sub.subject_id
ORDER BY s.name, sub.name;
SQLite
SQL ▾ 1 / 1 1 – 9 of 9
student_name subject_name score
Alice Johnson Chemistry 89
Alice Johnson Mathematics 85
Alice Johnson Physics 92
Bob Smith Chemistry 91
Bob Smith Mathematics 90
Bob Smith Physics 88
Charlie Brown Chemistry 84
Charlie Brown Mathematics 78
Charlie Brown Physics 80
SQL ▾ 1 / 1 1 – 3 of 3
subject_name average_score
Chemistry 88.0
Mathematics 84.3333333333333
Physics 86.6666666666667
SQL ▾ 1 / 1 1 – 9 of 9
student_name subject_name score
Alice Johnson Chemistry 89
Alice Johnson Mathematics 85
Alice Johnson Physics 92
Bob Smith Chemistry 91
Bob Smith Mathematics 90
Bob Smith Physics 88
Charlie Brown Chemistry 84
Charlie Brown Mathematics 78
Charlie Brown Physics 80
Go Live